

2026-05-05-p1-f12-multi-provider-backend-design

Phase 1 / Feature 12 — Multi-Provider Backend

Date: 2026-05-05 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 1 port from claude-code

1. Goal

Ship a unified multi-provider front-end for the existing `internal/llm.Provider` so users can choose Anthropic, AWS Bedrock, Google Vertex AI, or Azure OpenAI as the active LLM backend at startup. Selection happens through three precedence-ordered surfaces:

1. CLI flag `--provider <name>` (`anthropic`, `bedrock`, `vertex`, `azure`).
2. Env var `HELIX_LLM_PROVIDER`.
3. Interactive **tviz** wizard launched on first run when no provider is configured, and explicitly via the `helixcode wizard cobra` subcommand.

Model lists for every provider are sourced exclusively from `internal/verifier/` (LLMsVerifier) per CONST-036/037; no hardcoded `[]Model` literals. The four cloud-provider implementations (`anthropic_provider.go`, `bedrock_provider.go`, `azure_provider.go`, `vertexai_provider.go`) **already exist** under `helix_code/internal/llm/` — F12 wires them into a coherent factory + UX layer without re-implementing the providers.

The change is non-breaking: `cmd/cli/main.go` currently hardcodes Ollama via `llm.NewOllamaProvider`; after F12, Ollama remains the fallback when none of flag/env/wizard yields a configured cloud provider, preserving local-only workflows.

2. Architecture

The existing `llm.Provider` interface (`Generate/GenerateStream/GetModels/GetCapabilities/IsAvailable/GetHealth/Close/GetContextWindow/CountTokens`) is the unified contract. F12 adds three orthogonal pieces:

- **Selection layer** — `internal/llm/selector.go` resolves the active `ProviderType` using flag > env > config-file > wizard > Ollama-fallback precedence and returns a fully-constructed `llm.Provider`.
- **Factory entry** — extends `internal/llm/factory.go::NewProvider` with a thin `NewCloudProvider(ProviderConfigEntry) (Provider, error)` helper that validates the four F12 cloud types and rejects unknown ones with a descriptive error. The existing big switch is preserved.
- **Wizard** — `internal/llm/wizard.go` renders a `tviz.Application` with a `tviz.Form`, validating credentials inline and writing the resulting config into the

user-scoped config file (\$XDG_CONFIG_HOME/helixcode/config.yaml, mode 0600).

LLMsVerifier integration reuses `internal/llm/verifier_integration.go::VerifierModelSource`. The wizard’s “model” dropdown is populated by calling `VerifierModelSource.FetchModels(ctx)` and filtering by `ModelInfo.Provider == selectedProviderType`. When the verifier is unreachable, the wizard falls back to `verifier.fallback_models.go` and surfaces a yellow warning banner (“Verifier offline — using cached models”) rather than empty selectors. NO hardcoded model arrays.

`ProviderConfigEntry` (existing in `internal/llm/missing_types.go:94`) is the single config payload for all four cloud providers. F12 documents the field map per provider in §3.4.

3. Components

3.1 New files

- `helix_code/internal/llm/selector.go` — `Selector` struct, `Resolve(ctx, opts) (Provider, ProviderType, error)`, precedence rules, env var parsing.
- `helix_code/internal/llm/selector_test.go` — table-driven precedence + fallback tests.
- `helix_code/internal/llm/wizard.go` — tview application, form fields, validation, config write-out.
- `helix_code/internal/llm/wizard_test.go` — headless tview screen tests (`tcell.SimulationScreen`).
- `helix_code/internal/llm/wizard_writer.go` — file-only `WizardConfigWriter` so unit tests can assert without touching real `~/.config`.
- `helix_code/internal/llm/wizard_writer_test.go`.
- `helix_code/cmd/cli/wizard_cmd.go` — `helixcode wizard` cobra subcommand.
- `helix_code/cmd/cli/wizard_cmd_test.go`.
- `helix_code/tests/integration/multi_provider_test.go` — `// go:build integration, gated per §5`.
- `challenges/p1-f12-multi-provider/CHALLENGE.md + run.sh`.
- `helix_code/tests/integration/cmd/p1f12_challenge/main.go` — runtime evidence harness.

3.2 Modified files

- `helix_code/internal/llm/factory.go` — add `NewCloudProvider(cfg ProviderConfigEntry)` cloud-only helper that validates `cfg.Type ∈ {anthropic, bedrock, vertexai, azure}`. The existing `NewProvider` switch is left intact for non-cloud types.
- `helix_code/cmd/cli/main.go` — replace the hardcoded Ollama bootstrap in `NewCLI()` with `llm.NewSelector(...).Resolve(ctx, opts)`. Add `--provider`, `--model`, `--llm-config` top-level flags. Register `helixcode wizard` cobra command. First-run guard: when no config file exists AND no flag/env set, auto-launch the wizard before constructing the agent.

3.3 Types

```
// internal/llm/selector.go
type SelectorOptions struct {
    Flag      string // --provider value
    Env       string // HELIX_LLM_PROVIDER
    value
    ConfigPath string // ~/.config/helixcode/
    config.yaml
    Stdin      io.Reader // wizard input (for
    tests)
    Stdout     io.Writer // wizard output (for
    tests)
    AllowWizard bool // false in non-TTY
    contexts
    Verifier   *verifier.Adapter // optional; nil disables
    verifier-backed model lookup
}

type Selector struct{ opts SelectorOptions }

func NewSelector(opts SelectorOptions) *Selector
func (s *Selector) Resolve(ctx context.Context) (llm.Provider,
    llm.ProviderType, error)

// internal/llm/wizard.go
type WizardForm struct {
    Provider   ProviderType
    APIKey     string
    Model      string
    Region     string // bedrock, vertex
    ProjectID  string // vertex
    Endpoint   string // azure
    BaseURL    string // anthropic ANTHROPIC_BASE_URL override
}

type Wizard struct {
    app      *tview.Application
    screen   tcell.Screen // injected for tests
    writer   WizardConfigWriter
    verifier *verifier.Adapter
}

func NewWizard(w WizardConfigWriter, v *verifier.Adapter) *Wizard
func (w *Wizard) Run(ctx context.Context) (*WizardForm, error)
func (w *Wizard) RunHeadless(ctx context.Context, sim
    tcell.SimulationScreen) (*WizardForm, error)
```

```
// internal/llm/wizard_writer.go
type WizardConfigWriter interface {
    Write(form WizardForm) error
    Path() string
}

type FileWizardConfigWriter struct{ path string }
```

The unified Provider interface is unchanged. ProviderConfigEntry (existing) is reused; per-provider field expectations documented in §3.4.

3.4 ProviderConfigEntry field map (per provider)

Provider	Endpoint	APIKey	Parameters[...]
anthropic	optional ANTHROPIC_BASE_URL override	required	—
bedrock	unused	unused (uses AWS SDK chain)	region (string), profile (string)
vertexai	optional REST root override	unused (ADC via GOOGLE_APPLICATION_CREDENTIALS)	project_id, location
azure	required Azure resource endpoint	required	api_version, deployment

This mapping is canonical — implementations already use these keys in the existing *_provider.go files.

3.5 User surfaces

CLI flags (top-level; parsed before dispatcher):

```
- helixcode --provider anthropic -
helixcode --provider bedrock --model claude-3-5-sonnet-20241022 -
helixcode --llm-config /path/to/config.yaml
```

Env vars:

```
- HELIX_LLM_PROVIDER (one of anthropic, bedrock, vertex, azure) -
ANTHROPIC_API_KEY, ANTHROPIC_BASE_URL - AWS_ACCESS_KEY_ID,
AWS_SECRET_ACCESS_KEY, AWS_REGION - GOOGLE_APPLICATION_CREDENTIALS,
VERTEX_PROJECT_ID, VERTEX_LOCATION - AZURE_OPENAI_API_KEY,
AZURE_OPENAI_ENDPOINT, AZURE_OPENAI_API_VERSION
```

Cobra:

```
- helixcode wizard — explicit re-run of the configuration wizard (overwrites prior
config).
```

First-run trigger: in cmd/cli/main.go startup, if os.Stat(configPath) returns os.ErrNotExist AND no --provider flag AND no HELIX_LLM_PROVIDER env var AND term.IsTerminal(int(os.Stdin.Fd())) returns true, the selector auto-launches Wizard.Run. Non-TTY (CI, scripts) bypasses the wizard and falls back to Ollama with a stderr WARN.

4. Data flow

4.1 Startup precedence

```
NewCLI()
└─ Selector.Resolve(ctx)
    └─ if opts.Flag != "":      type := opts.Flag
        └─ else if opts.Env != "": type := opts.Env
            └─ else if configFile.exists: type := configFile.Provider
                └─ else if AllowWizard && tty: type, form := Wizard.Run() ; writer.
                    └─ else: type := ProviderTypeOllama (fallback)
                        └─ cfg := buildProviderConfigEntry(type, env, configFile)
                            └─ provider, err := llm.NewProvider(cfg) // existing factory
```

4.2 Wizard flow (tview)

```
Wizard.Run
└─ render tview.Form
    └─ DropDown "Provider" → [anthropic, bedrock, vertex, azure]
        └─ on-change: rebuild model dropdown via VerifierModelSource.FetchM
            filtered by ProviderType
        └─ InputField "API Key" / "Region" / "Project ID" / "Endpoint" (con
        └─ Buttons [Save] [Cancel]
    └─ on-Save: validate (non-empty mandatories per §3.4), then writer.Write
    └─ on-Cancel: return ErrWizardCancelled
    └─ return WizardForm
```

4.3 Model discovery

```
provider.GetModels()
└─ existing impl: returns provider.models slice
    └─ which is populated at construction by:
        VerifierModelSource.FetchModels(ctx) // CONST-036/037
        └─ filter by ProviderType
```

If the verifier is unreachable at provider construction, `verifier.fallback_models.go` supplies the cached set. If both fail, the provider construction returns a non-nil error and the selector reports the failure to the user with the path to the wizard for re-configuration.

5. Error handling, edge cases, and anti-bluff

5.1 Error paths

- **Missing credentials** — provider construction returns a typed error (`ErrProviderUnavailable` from `missing_types.go:187`); selector surfaces via `fmt.Errorf("provider %s: %w", type, err)` and tells the user to run `helixcode wizard`.
- **Verifier unreachable** — fall back to `fallback_models.go`; emit one `stderr WARN`; do not crash.
- **Wizard cancelled** — exit 1 with a friendly message; do not write a partial config file.
- **Invalid endpoint URL** — wizard shows inline error; form does not advance.

- **Non-TTY with wizard required** — selector returns an explicit error: “no provider configured and stdin is not a TTY; set `--provider, HELIX_LLM_PROVIDER`, or run `helixcode wizard` interactively”.
- **Concurrent first-run launches** — wizard writer uses `os.OpenFile(path, O_CREATE|O_EXCL|O_WRONLY, 0600)`; second concurrent run returns `os.ErrExist` and the user is told to use `helixcode wizard` to overwrite.

5.2 Anti-bluff (CONST-035 / §11.9) — LOUD

The single most dangerous bluff vector for F12 is fake-PASS on cloud calls. Real Bedrock/Vertex/Azure round-trips require credentials we cannot assume the test environment has. Therefore:

1. **Unit tests** — mocks of the cloud-SDK boundary are permitted (per Universal Rule 2). They **MUST** validate request shaping (URL, headers, body), response parsing, and error mapping.
2. **Integration tests** (`-tags=integration`) — gated by env-var presence, with explicit SKIP-OK markers when credentials are missing:

- `ANTHROPIC_API_KEY` → `TestProvider_Anthropic_*`
- `AWS_ACCESS_KEY_ID` && `AWS_REGION` → `TestProvider_Bedrock_*`
- `GOOGLE_APPLICATION_CREDENTIALS` → `TestProvider_Vertex_*`
- `AZURE_OPENAI_API_KEY` && `AZURE_OPENAI_ENDPOINT` → `TestProvider_Azure_*`

Skip body **MUST** be exactly `t.Skip("SKIP-OK: P1-F12 cloud creds not provided")` — bare skips break `make ci-validate-all`.

3. **Challenge harness** must clearly separate evidence:

- **Local evidence (always runs)**: factory construction of all four `ProviderTypes` with synthetic configs (no network); wizard form rendering + field validation under `tcell.SimulationScreen`; selector precedence (`flag > env > config > wizard > fallback`); LLMsVerifier-backed model dropdown population using a stub adapter pointing at the embedded verifier server.
- **Credentialed evidence (skipped when env unset)**: real `Health(ctx)` round-trip per provider, with the SKIP markers above. The Challenge prints a final breakdown — `LOCAL: PASS` vs `CLOUD: SKIPPED (creds missing)` — so a reader can never confuse the two.

4. `internal/llm/wizard.go` **must use real** `tview.Application` wired via `WithScreen(tcell.SimulationScreen)` for tests. Tests assert on rendered cell content, not on captured `fmt.Print` output. Form-state assertions exercise the real callback chain.

5. The standard 4-term smoke clean check applies to every new file:

```
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  internal/llm/selector.go internal/llm/wizard.go internal/
  llm/wizard_writer.go \
  cmd/cli/wizard_cmd.go && echo BLUFF || echo clean
```

6. Testing

Unit (mocks OK): - TestSelector_FlagWinsOverEnv -
TestSelector_EnvWinsOverConfig - TestSelector_ConfigWinsOverWizard -
TestSelector_WizardWhenNothingSet_TTY -
TestSelector_OllamaFallback_NonTTY -
TestSelector_RejectsUnknownProvider -
TestNewCloudProvider_AcceptsAllFour -
TestNewCloudProvider_RejectsNonCloudType -
TestWizard_RendersAllFourProviderOptions (tcell.SimulationScreen) -
TestWizard_DynamicModelDropdown_ViaVerifier -
TestWizard_VerifierOffline_FallbackBannerShown -
TestWizard_ValidatesMandatoryFieldsPerProvider -
TestWizard_CancelReturnsError_NoFileWritten -
TestWizardWriter_File_0_EXCL_PreventsOverwrite -
TestWizardWriter_File_Mode0600 -
TestWizardCmd_RunsWizard_AndPersists - Per-provider request-shaping tests
(TestAnthropicProvider_Generate_ShapesRequest, etc.) — these may already exist; F12 only adds gaps it identifies.

Integration (-tags=integration, gated): -
TestMultiProvider_Anthropic_Health — SKIP-OK: P1-F12 cloud creds not provided when env unset. - TestMultiProvider_Bedrock_Health -
TestMultiProvider_Vertex_Health - TestMultiProvider_Azure_Health -
TestMultiProvider_VerifierBackedModelList — runs against the embedded verifier server (always available; not credential-gated).

Challenge: - Local: factory + wizard rendering + selector precedence + verifier-backed model list (always runs). - Credentialed: real Health(ctx) per provider (skipped when creds missing, with explicit reporting in the Challenge output).

7. Cross-platform

All cloud SDKs are pure Go and already in helix_code/go.mod: - github.com/aws/aws-sdk-go-v2 v1.32.7 and service/bedrockruntime v1.23.1 - github.com/Azure/azure-sdk-for-go/sdk/azcore v1.16.0, sdk/azidentity v1.8.0 - Vertex AI is implemented over **raw HTTPS** + golang.org/x/oauth2/google (already in deps; see vertexai_provider.go); we deliberately do NOT pull cloud.google.com/go/aiplatform because the existing impl already works and adding the SDK would be a 30+MB dep churn for zero gain. - github.com/rivo/tview v0.42.0 and github.com/gdamore/tcell/v2 are already in deps.

No new dependencies required. Cross-compile sanity for linux/darwin/windows is the canonical check; tview supports all three.

8. Out of scope (deferred)

- Multi-provider routing / fallback chains across providers (route-by-cost, route-by-availability).
- Per-request provider switching mid-session.
- Encrypted credential storage (creds remain in .env mode 0600 + system OS key-store integration deferred).

- Telemetry attribution per provider (call counts, cost dashboards).
- OAuth interactive flow for Anthropic Claude.ai accounts (only API-key path supported in F12).
- Vertex AI Anthropic-via-Model-Garden enforcement (existing `vertexai_provider.go` supports it; F12 does not change behaviour).

9. Constitutional compliance

- **§11.9 / CONST-035** — Challenge separates LOCAL vs CLOUD evidence with explicit credential-gating, never claims a green PASS without runtime evidence.
- **CONST-036 / CONST-037** — Wizard model dropdown and provider construction both source models from `internal/verifier/`. Hardcoded model lists are forbidden and asserted-against by `TestWizard_DynamicModelDropdown_ViaVerifier`.
- **CONST-038** — Existing verifier poller (60s interval) is honoured; F12 does not introduce any cache that could outlive the 60s mandate.
- **CONST-039** — All four mandated cloud providers shipped together; `TestNewCloudProvider_AcceptsAllFour` enforces.
- **CONST-040** — Capability flags propagate through `VerifierModelSource.ConvertVerifiedToModelInfo`; never hardcoded.
- **CONST-042** — Credentials are read from `env` and `~/ .config/helixcode/ config.yaml` (mode 0600, in `.gitignore` — verified by Challenge). Wizard writer enforces mode 0600 via `os.OpenFile` with explicit perm bits.
- **CONST-043** — All commits land on `main` non-force; the close-out task pushes to all four remotes non-force.

10. Open questions resolved

Q	Answer
Q1: provider scope	(A) ship all four — Anthropic, AWS Bedrock, Google Vertex AI, Azure OpenAI
Q2: selection surface	(C) <code>--provider</code> flag + <code>HELIX_LLM_PROVIDER</code> env + interactive wizard (<code>cobra helixcode wizard</code> + first-run auto-launch)
Q3: relationship to existing internal/llm	(B) wrap existing <code>internal/llm</code> (4 cloud impls already exist); F12 adds the unified selection/factory/wizard layer; non-breaking
Q4: wizard UX	(B) full tview TUI (modal forms, focus, inline validation) — NOT plain stdin prompts
Q5: model source	(A) <code>LLMsVerifier</code> mandatory; <code>internal/verifier</code> is the sole source of truth for model lists per CONST-036/037